FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Optimización de Consultas SQL

Gestión de Bases de Datos

Rodrigo Pavez Madariaga

rpavez@utalca.cl

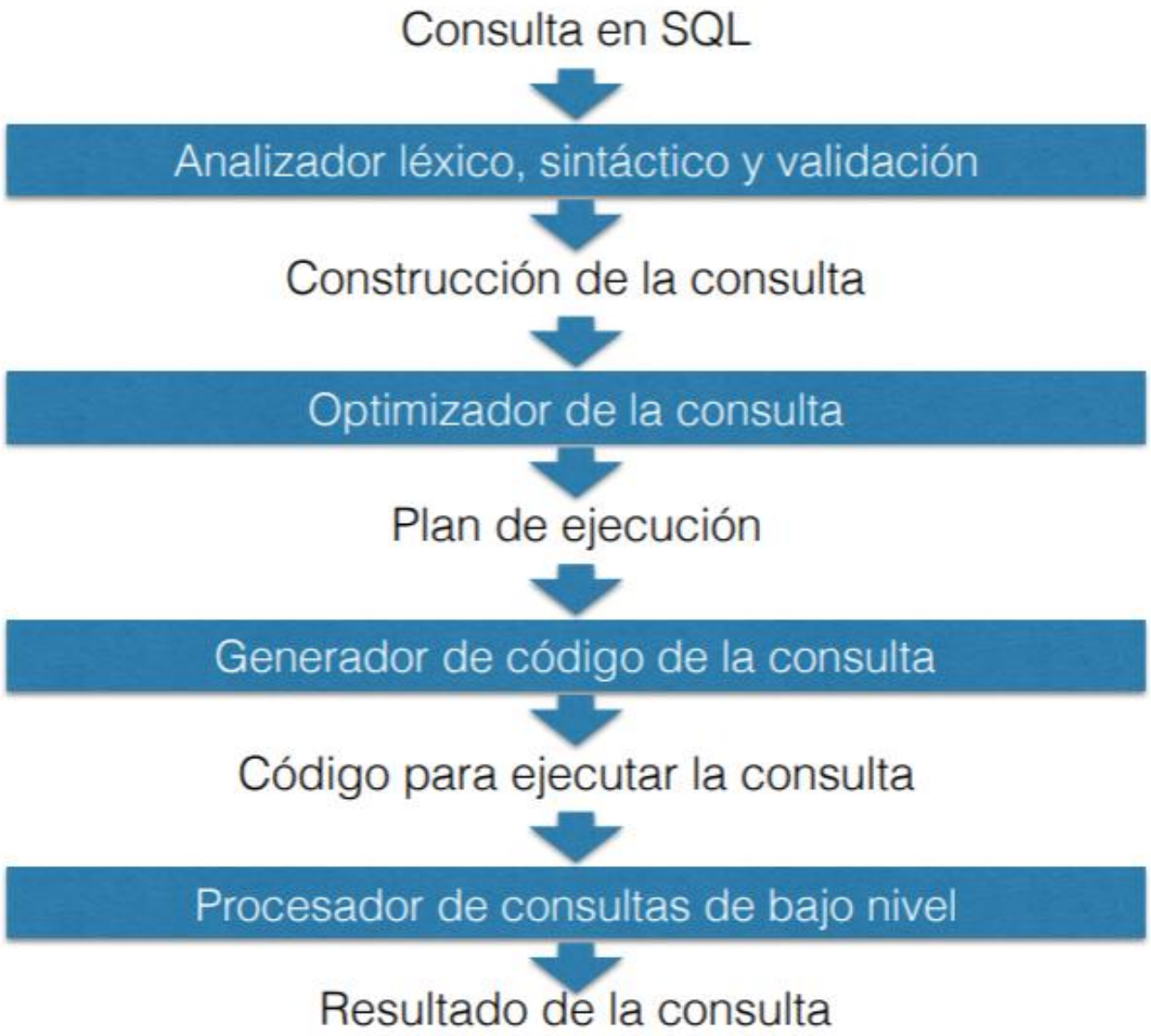
INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Procesamiento de una consulta

La ejecución de una consulta SQL tiene una secuencia que es transparente al usuario y al desarrollador, ya que es atendida y resuelta por el Sistema Gestor de Bases de Datos.



INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Consulta SQL

```
SELECT T1.B, T2.D
FROM T1, T2
WHERE
T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

Consulta SQL y sus operadores

SELECT T1.B, T2.D ← Proyección
FROM T1, T2 ← Producto Cartesiano
WHERE
T1.C = T2.C **AND** T1.A="3" **AND** T2.E="1"

Join

Selección

Consulta SQL y el álgebra relacional

SELECT T1.B, T2.D ← Proyección
FROM T1, T2 ← Producto Cartesiano
WHERE
T1.C = T2.C **AND** T1.A="3" **AND** T2.E="1"

Join

Selección

$$\Pi_{T1.B,\ T2.D}\ (\sigma_{(T1.C = T2.C\ \wedge\ T1.A=\text{“3”}\ \wedge\ T2.E=\text{“1”})}(T1 \times T2))$$

Selección: $\sigma_c(R)$

- Búsqueda lineal (fuerza bruta)
- Búsqueda binaria (sobre un fichero ordenado)
- Utilización de un índice primario (hash)
- Utilización de un índice agrupado (son un atributo que no es clave)
- Utilización de un índice secundario (B+) sobre una comparación de igualdad

Proyección: $\Pi_{A1,\ldots,An}(R)$

- Sencillo si los atributos proyectados incluyen una clave … ya que no habrá tuplas repetidas.
- Caso contrario, se tendrá que eliminar repetidos (si se usa DISTINCT)
- Eliminar repetidos puede ser costoso
  - Opción 1: ordenar y eliminar
  - Opción 2: usar una función de dispersión (hash)

Join: $R \bowtie_{A=B} S$

- Join de bucle anidado: fuerza bruta, ***N x M***
- Join de bucle simple: si existe un índice hash en ***R.A*** o ***S.B***
- Join de ordenación-mezcla: si los registros de ***R*** y ***S*** están ordenados
- Join usando dispersión: si ***R*** y ***S*** comparten el fichero de dispersión

Operaciones de conjuntos

- Producto cartesiano (R x S) es muy costoso (en tiempo y espacio)
- UNION, INTERSECTION y EXCEPT
  - ordenación y mezcla
  - función de dispersión

Ejecución de una consulta

```
SELECT  T1.B, T2.D
FROM  T1, T2
WHERE T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

T1

A	B	C
1	a	10
2	a	20
3	b	10
4	b	30
5	c	40

T2

C	D	E
10	x	1
20	y	1
30	y	1
40	z	0
50	x	1

Ejecución de una consulta

```
SELECT  T1.B, T2.D
FROM  T1, T2
WHERE T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

T1

A	B	C
1	a	10
2	a	20
3	b	10
4	b	30
5	c	40

JOIN

T2

C	D	E
10	x	1
20	y	1
30	y	1
40	z	0
50	x	1

Cómo ejecutar la consulta?

```
SELECT  T1.B, T2.D
FROM  T1, T2
WHERE T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

Opción 1

- Realizar producto cartesiano de T1 y T2
- Aplicar las condiciones
- Proyectar

Cómo ejecutar la consulta?

```
SELECT  T1.B, T2.D
FROM  T1, T2
WHERE T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

T1.A	T1.B	T1.C	T2.C	T2.D	T2.E
1	a	10	10	x	1
1	a	10	20	y	1
...	...	...	...	...	...
3	b	10	10	x	1
...	...	...	...	...	...
5	c	40	40	z	0
5	c	40	50	x	1

Problemas...

```
SELECT  T1.B, T2.D
FROM  T1, T2
WHERE T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

- Producto cartesiano es una operación ***extremadamente costosa***
- El orden de evaluación de las condiciones afecta el tiempo de ejecución
- No se están empleando índices

Conclusión: Ejecución lenta / Tiempo de respuesta alto

¿Se puede mejorar?

Para eso se debe realizar una optimización basada en heurísticas.

Árbol de Consultas

- Equivale a una expresión de Algebra Relacional.
- Los nodos hoja contienen a las tablas y los nodos internos las operaciones del algebra relacional.
- Un árbol representa un plan de ejecución (evaluación bottom-up)

Árbol de Consultas

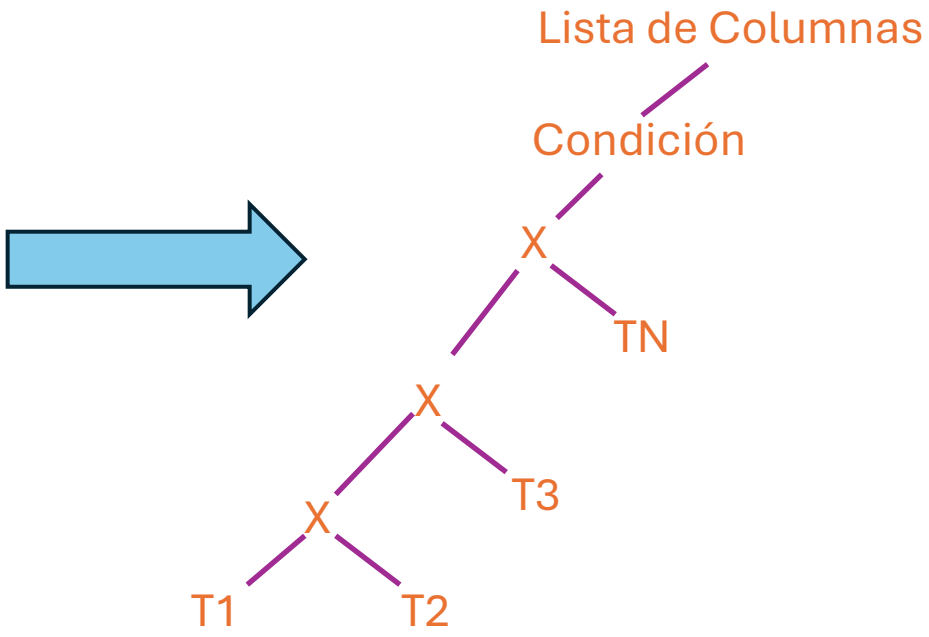
Representa una Consulta de álgebra relacional. Un árbol de consultas es un ***Árbol Lineal Izquierdo***, donde cada nodo binario corresponde a un producto cartesiano. La selección es sobre toda la condición y la proyección sobre todos los atributos. Cada hijo derecho es una tabla. **Define como se ejecuta una consulta**.

Ejemplo con la sintaxis

SELECT (Lista de columnas)

FROM T1, T2, ..., TN

WHERE (Condición(es))



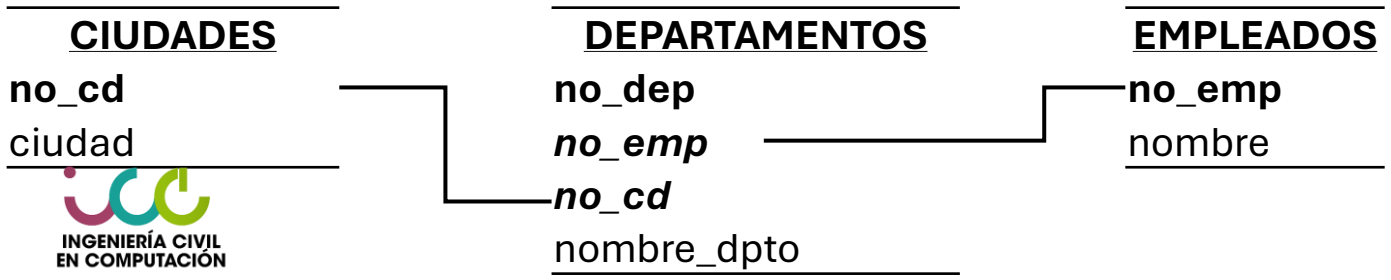
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Ejemplo de árbol de Ejecución de una Consulta

CIUDADES	
no_cd	ciudad
20	Curicó
21	Talca
32	Linares
45	Chillán
47	Concepción
48	Los Angeles
52	Temuno

DEPARTAMENTOS			
no_dep	***no_emp***	***no_cd***	nombre_dpto
50	**1002**	**20**	Desarrollo
100	**1006**	**32**	Finanzas
150	**1007**	**47**	Calidad
200	**1009**	**45**	RRHH
50	**1001**	**48**	Desarrollo
50	**1010**	**52**	Desarrollo
100	**1012**	**21**	Finanzas
150	**1004**	**20**	Calidad

EMPLEADOS	
no_emp	nombre
1001	Karina
1002	Lorenzo
1003	Cristobal
1004	José
1005	Guillermo
1006	Camila
1007	Ximena
1008	Rossana
1009	Josefa
1010	Cristian
1011	Christofer
1012	Carlos



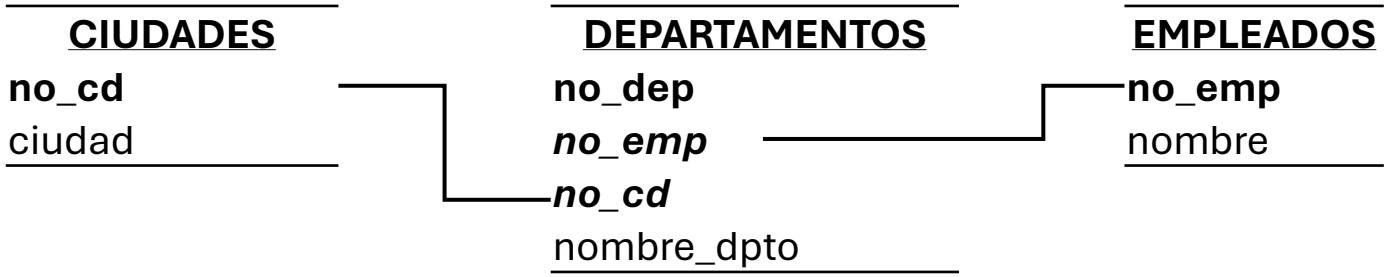
INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Ejemplo de árbol de Ejecución de una Consulta

Consultamos información de tres tablas:

```
SELECT no_emp, nombre, nombre_dpto, ciudad
FROM EMPLEADOS e, DEPARTAMENTOS d, CIUDADES c
WHERE (e.no_emp = d.no_emp AND d.no_cd=c.no_cd)
AND d.no_dep = 50;
```



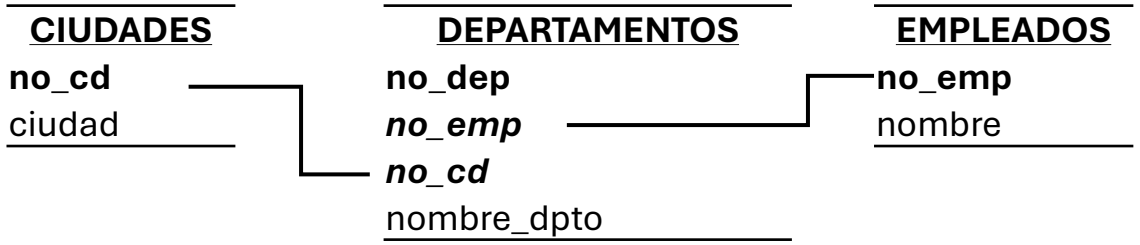
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

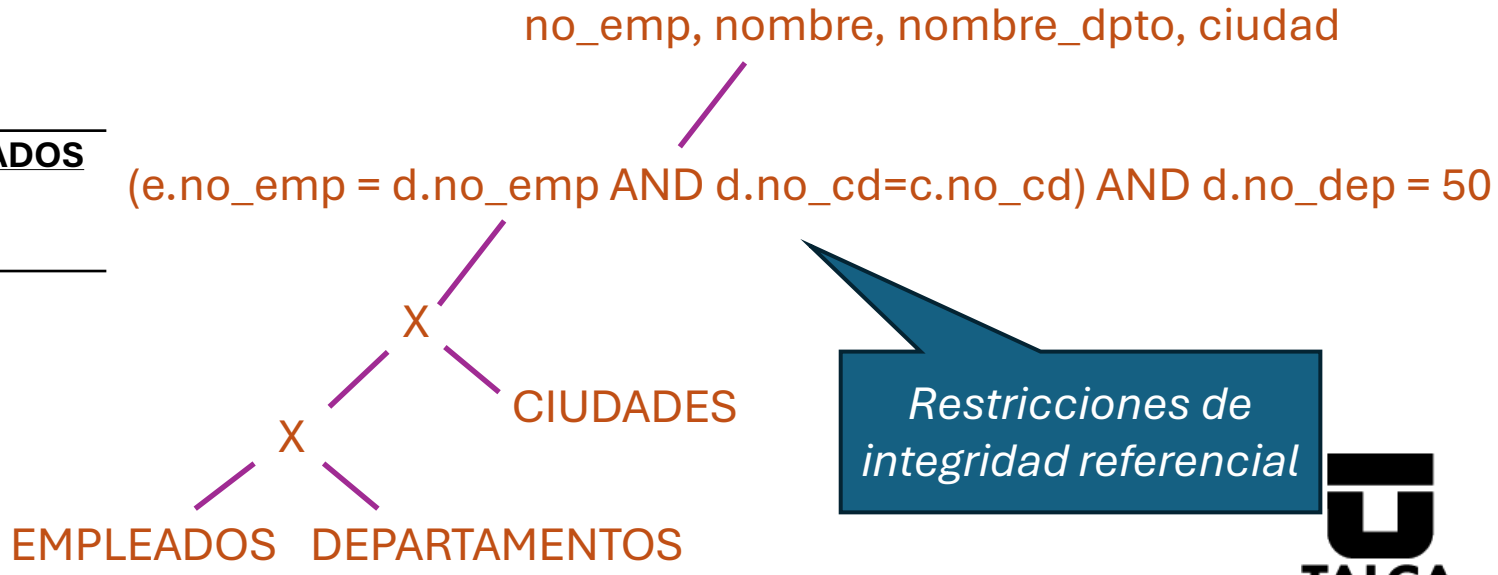
Ejemplo de árbol de Ejecución de una Consulta

Consultamos información de tres tablas:

```
SELECT no_emp, nombre, nombre_dpto, ciudad
FROM EMPLEADOS e, DEPARTAMENTOS d, CIUDADES c
WHERE (e.no_emp = d.no_emp AND d.no_cd=c.no_cd)
AND d.no_dep = 50;
```

Solución 1





Restricciones de integridad referencial

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

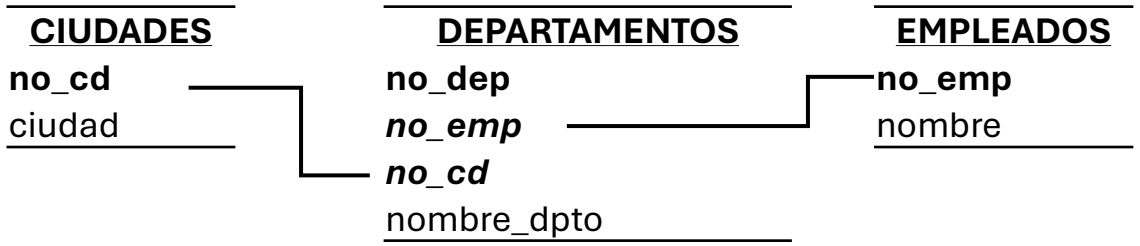
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

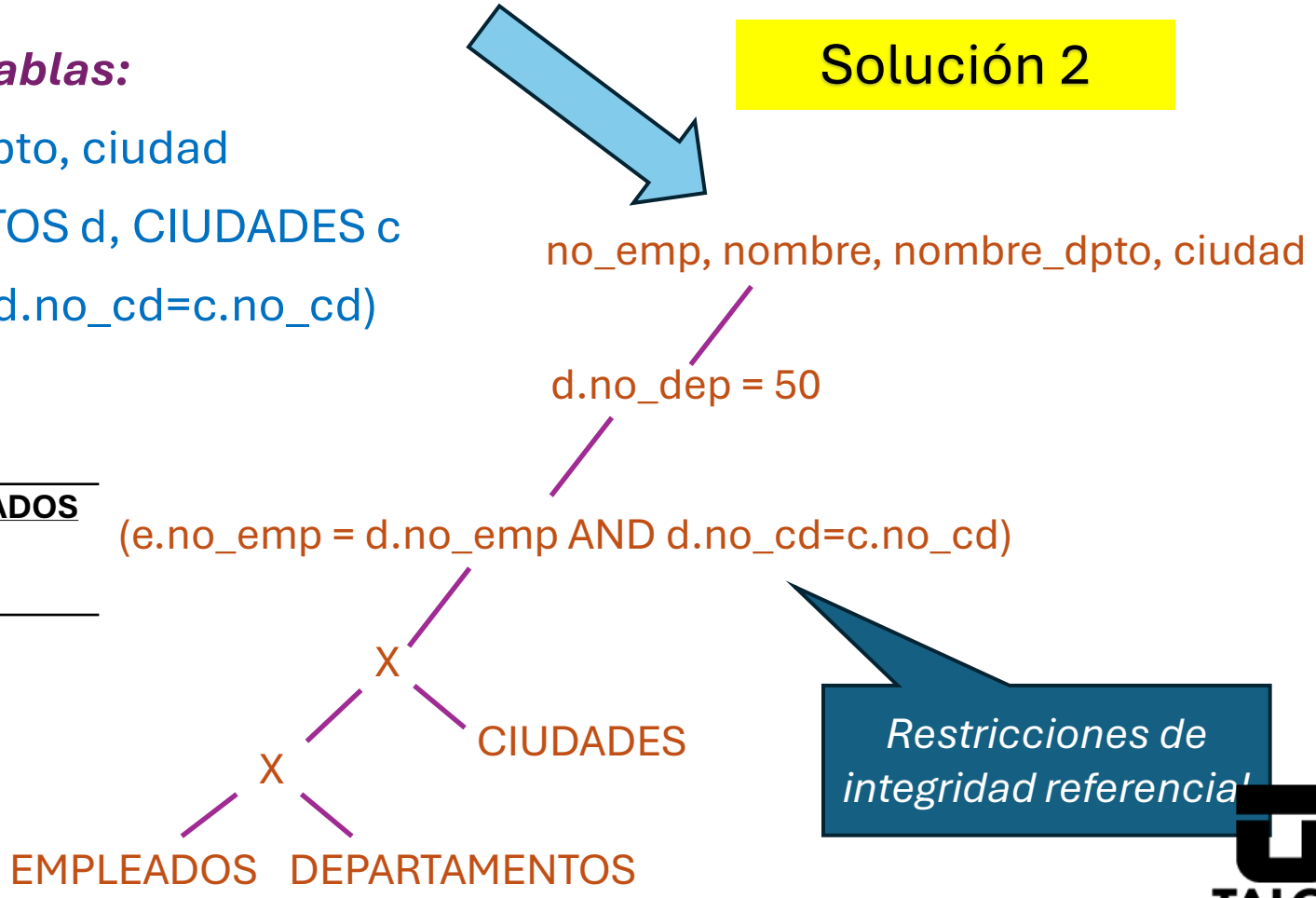
Ejemplo de árbol de Ejecución de una Consulta

Consultamos información de tres tablas:

```
SELECT no_emp, nombre, nombre_dpto, ciudad
FROM EMPLEADOS e, DEPARTAMENTOS d, CIUDADES c
WHERE (e.no_emp = d.no_emp AND d.no_cd=c.no_cd)
AND d.no_dep = 50;
```

Solución 2





INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Ejemplo de árbol de Ejecución de una Consulta

Consultamos información de tres tablas:

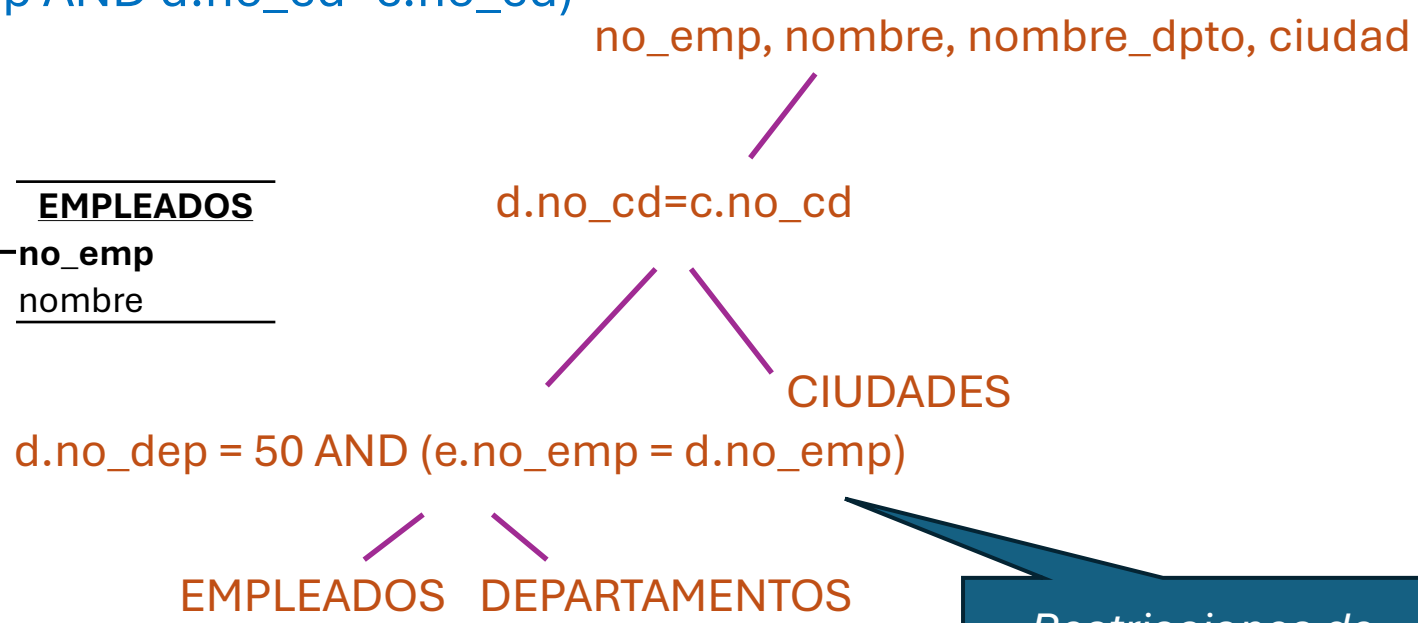
SELECT no_emp, nombre, nombre_dpto, ciudad
FROM EMPLEADOS e, DEPARTAMENTOS d, CIUDADES c
WHERE (e.no_emp = d.no_emp AND d.no_cd=c.no_cd)
AND d.no_dep = 50;

Solución 2 Optimizada

CIUDADES
no_cd
ciudad

DEPARTAMENTOS
no_dep
no_emp
no_cd
nombre_dpto

EMPLEADOS
no_emp
nombre



Restricciones de integridad referencial

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Pregunta...

¿Cómo podemos verificar que las distintas "***optimizaciones***" generan mejoras sobre la consulta original?

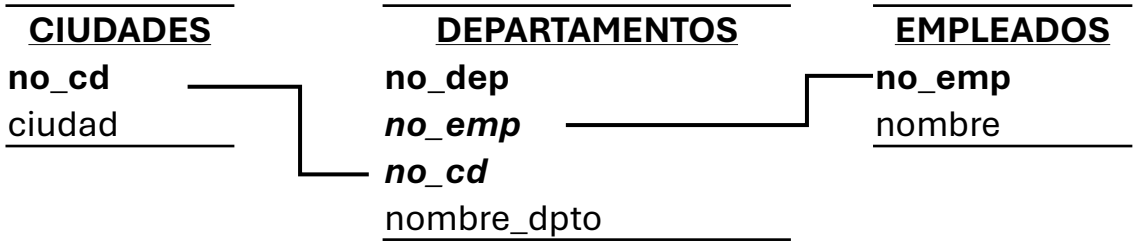
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

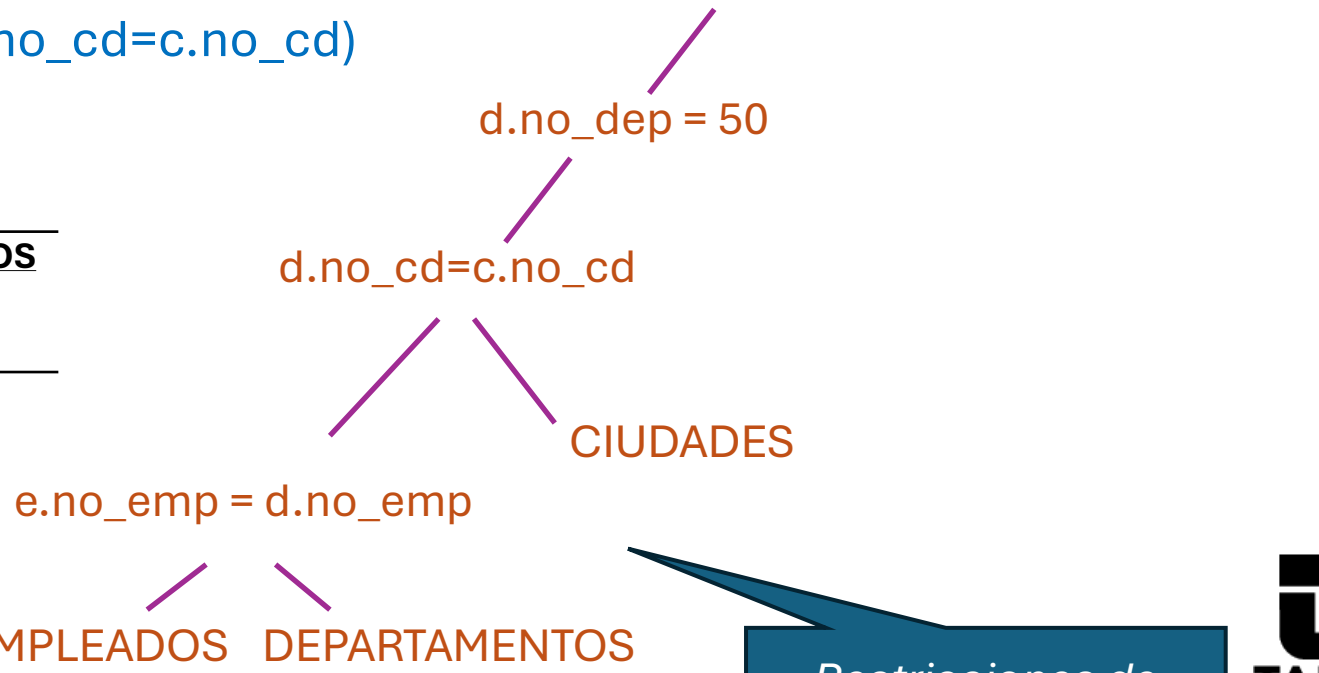
Ejemplo de árbol de Ejecución de una Consulta

Consultamos información de tres tablas:

```
SELECT no_emp, nombre, nombre_dpto, ciudad
FROM EMPLEADOS e, DEPARTAMENTOS d, CIUDADES c
WHERE (e.no_emp = d.no_emp AND d.no_cd=c.no_cd)
AND d.no_dep = 50;
```

Solución 2 Optimizada





Restricciones de integridad referencial

INGENIERÍA CIVIL EN COMPUTACIÓN

Eliminar y aplicar las restricciones de integridad referencial al inicio

TALCA UNIVERSIDAD CHILE

Reglas de Optimización de Consultas

Las **reglas de optimización de consultas** son transformaciones basadas en el **álgebra relacional** que permiten convertir un árbol de consulta en otro equivalente, pero **más eficiente en ejecución** (sin cambiar el resultado).

Trabajo autónomo

Parte I:

Explore el libro Fundamentos de bases de datos (Abraham Silberschatz, Henry F. Korth 4ta edición) en el **Capítulo 13: Procesamiento de Consultas (hasta el punto 13.4)** y luego desarrolle las actividades presentadas a continuación.

- ¿Cómo podemos medir el tiempo o rendimiento de una consulta?
- Describa el comportamiento de 2 algoritmos básicos.
- Respecto a la selección con índices, describa alguna situación especial donde se logre un mejor resultado.
- ¿Qué efecto tiene en el rendimiento el realizar consultas con Conjunciones o Disyunciones?

Parte II:

Explore el libro Fundamentos de sistemas de bases de datos (Ramez Elmasri, Shamkant B. Navathe) - 5ta edición en la ***Sección 15.3 Algoritmos para las operaciones SELECT y JOIN*** del **Capítulo 15: Algoritmos para procesamiento y optimización de consultas** y realice un mapa conceptual respecto a las técnicas que se abordan.

Optimización basada en heurísticas

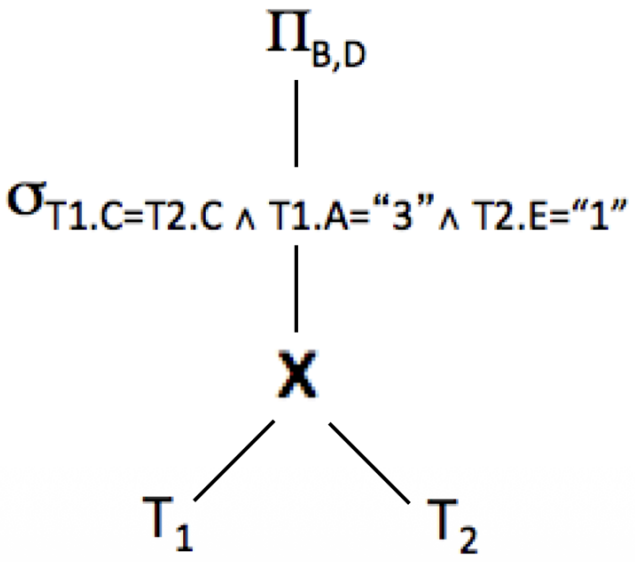
Retomando... una consulta SQL …

```
SELECT  T1.B, T2.D
FROM  T1, T2
WHERE T1.C = T2.C AND T1.A="3" AND T2.E="1"
```

se traduce en …

$$\Pi_{B,D}\,(\,\sigma_{T1.C=T2.C \,\wedge\, T1.A=\text{“3”} \,\wedge\, T2.E=\text{“1”}}\,(T_1 \times T_2))$$

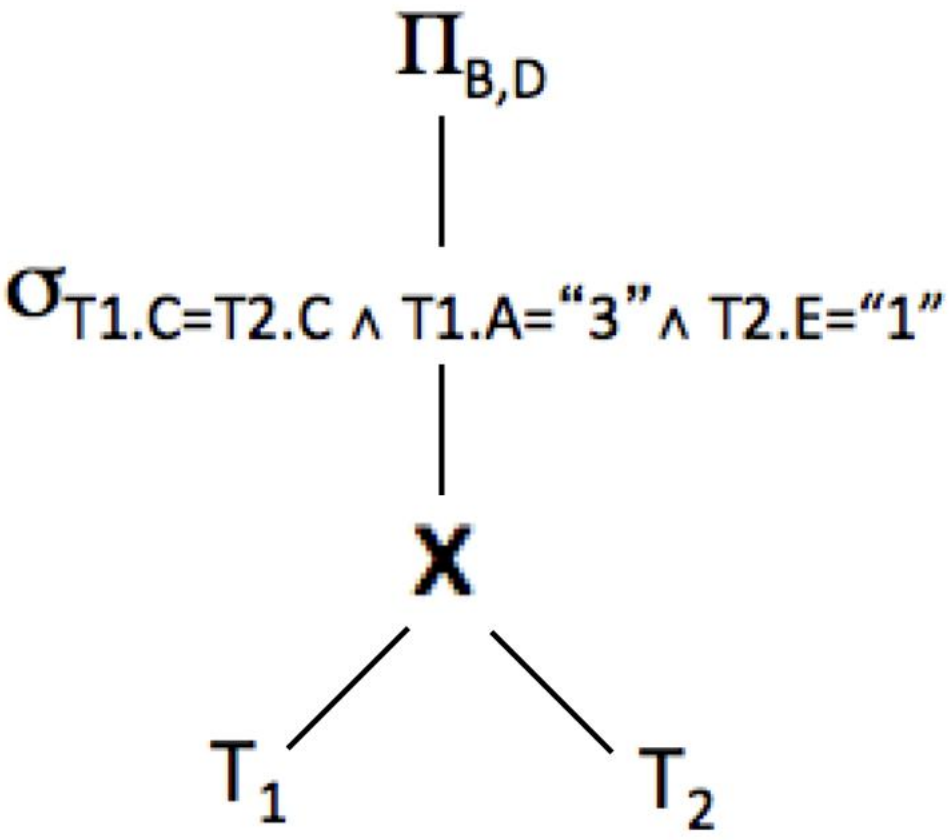
y se estudia como …



Optimización basada en heurísticas

Árbol de consulta

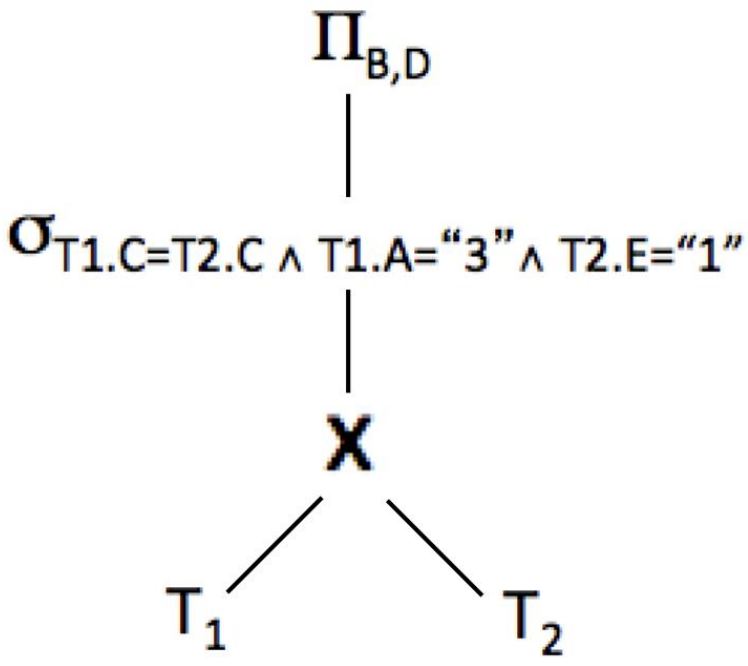
- Equivale a una expresión de Algebra Relacional
- Los nodos hoja contienen a las tablas y los nodos internos las operaciones del algebra relacional
- Un árbol representa una plan de ejecución (evaluación bottom-up)

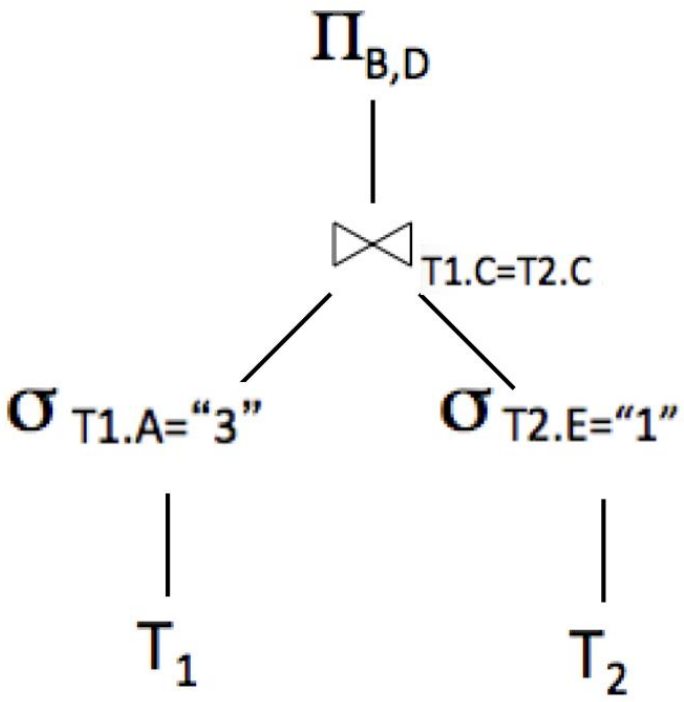


Optimización basada en heurísticas

Árbol de consulta

Una expresión de A.R. puede tener múltiples árboles de consulta equivalentes.





Optimización basada en heurísticas

Árbol de consulta

- Permite estudiar y definir métodos de optimización de consultas SQL
- Optimización basada en heurísticas: emplea reglas de equivalencia entre operaciones
- Optimización basada en coste: emplea estadísticas sobre las tablas y los resultados intermedios

Reglas heurísticas básicas

- Aplicar en primer lugar las operaciones que reducen el tamaño de los resultados intermedios (es decir, selecciones y proyecciones).
- Re-ordenar los nodos hojas del árbol para que las operaciones de selección y ***join*** más restrictivas sean ejecutadas antes que otras operaciones.
- Usar ***join*** en lugar de producto cartesiano combinado con una selección.

Reglas generales de transformación (1)

Cascada de selecciones

$$\sigma_{c1 \text{ AND } c2 \text{ AND } \ldots \text{ AND } cn}(R) \equiv \sigma_{c1}(\sigma_{c2}(\ldots(\sigma_{cn}(R))\ldots))$$

Conmutatividad de la selección

$$\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$$

Cascada de proyecciones

$$\pi_{\text{Lista1}}(\pi_{\text{Lista2}}(\ldots(\pi_{\text{Listan}}(R))\ldots)) \equiv \pi_{\text{Lista1}}(R)$$

Reglas generales de transformación (2)

Conmutación de selección por proyección

$$\pi_{A1,\,A2,\,\ldots,\,An}(\sigma_c(R)) \equiv \sigma_c(\pi_{A1,\,A2,\,\ldots,\,An}(R))$$

Conmutatividad de join y producto cartesiano

$$R \bowtie_c S \equiv S \bowtie_c R \qquad\qquad R \times S \equiv S \times R$$

Conmutación de selección con join

$$\sigma_c(R \bowtie S) \equiv (\sigma_c(R)) \bowtie S$$

$$\sigma_c(R \bowtie S) \equiv (\sigma_{c1}(R)) \bowtie (\sigma_{c2}(S))$$

Reglas generales de transformación (3)

Conmutación de proyección con join

$$\pi_L (R \bowtie_c S) \equiv (\pi_{A1, \ldots, An} (R)) \bowtie_c (\pi_{B1, \ldots, Bm} (S))$$

Conmutatividad de las operaciones de unión e intersección

$$R \cup S \equiv S \cup R$$

Asociatividad de join, producto cartesiano, unión e intersección

$$(R \; \theta \; S) \; \theta \; T \equiv R \; \theta \; (S \; \theta \; T)$$

Reglas generales de transformación (4)

Conmutación de selección con operación de conjunto

$$\sigma_c (R \ \theta \ S) \equiv (\sigma_c (R)) \ \theta \ (\sigma_c (S))$$

Conmutación de proyección con unión

$$\pi_L (R \cup S) \equiv (\pi_L (R)) \cup (\pi_L (S))$$

Conversión de selección + producto cartesiano en join

$$(\sigma_c (R \times S)) \equiv (R \bowtie_c S)$$

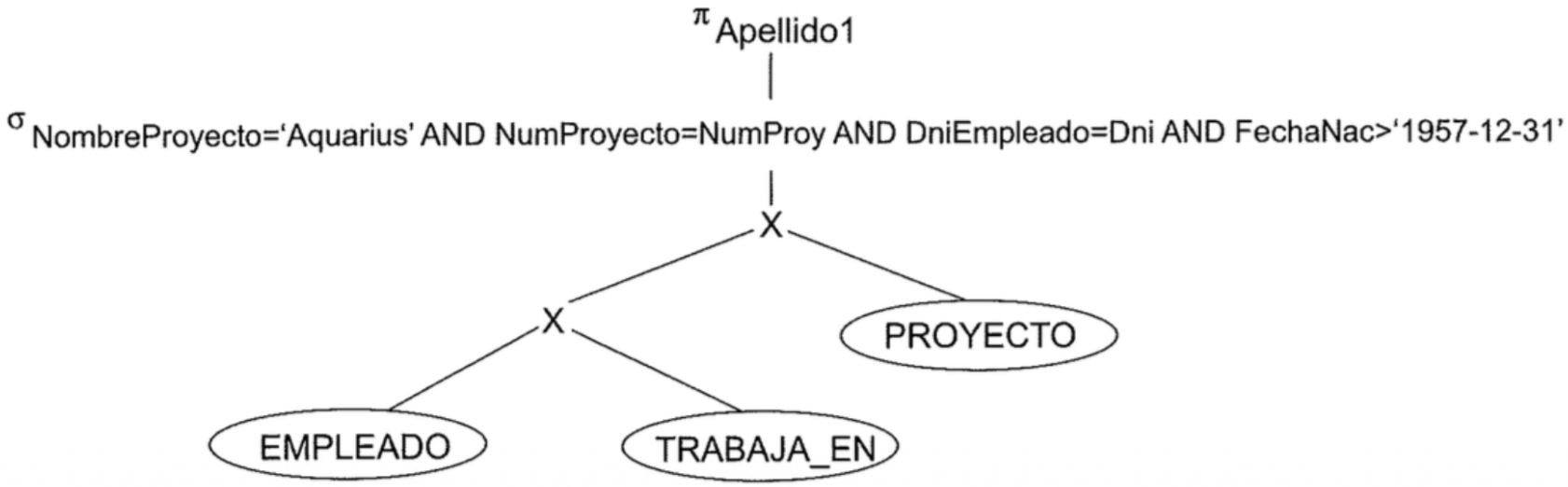
Algoritmo básico de optimización

1. Descomponer selecciones conjuntivas en una cascada de operaciones y moverlas lo más abajo posible en el árbol
2. Re-ordenar las relaciones de los nodos hoja
   - Las relaciones con selecciones más restrictivas deben ejecutarse primero.
   - Asegurarse que la ordenación no generar productos cartesianos.
3. Transformar las combinaciones selección + producto cartesiano en joins siempre que sea posible.
4. Descomponer las operaciones de proyección y desplazar las sub-proyecciones lo más abajo posible en el árbol.
5. Identificar sub-árboles que representan grupos de operaciones que se pueden ejecutar mediante un único algoritmo.

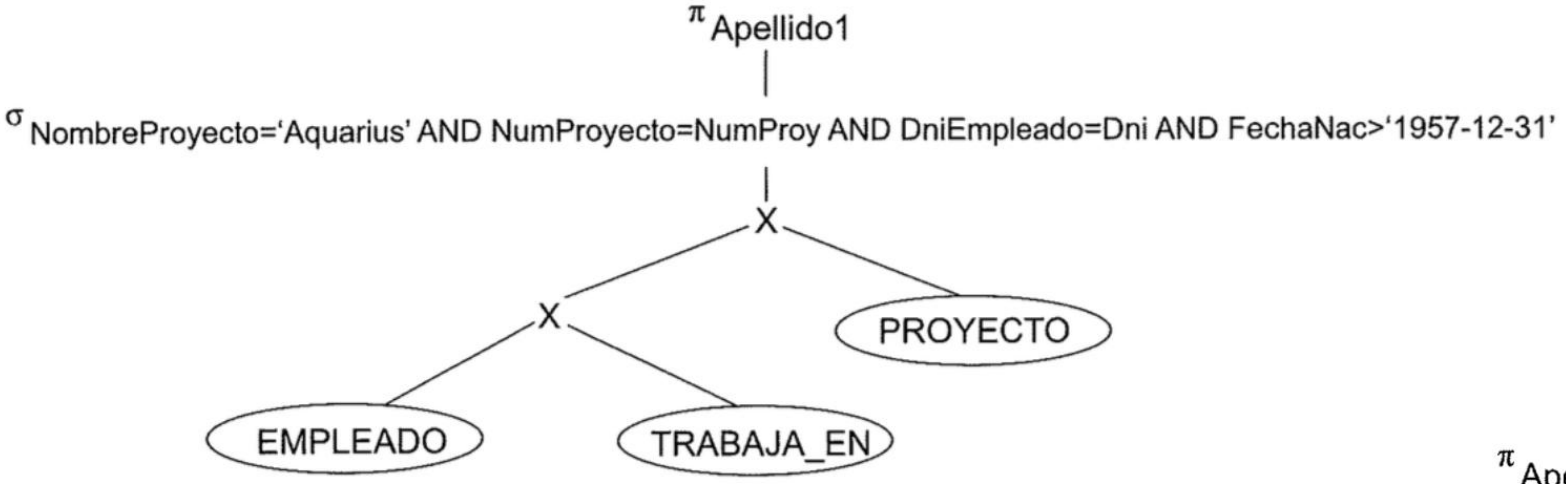
Ejemplo de transformación

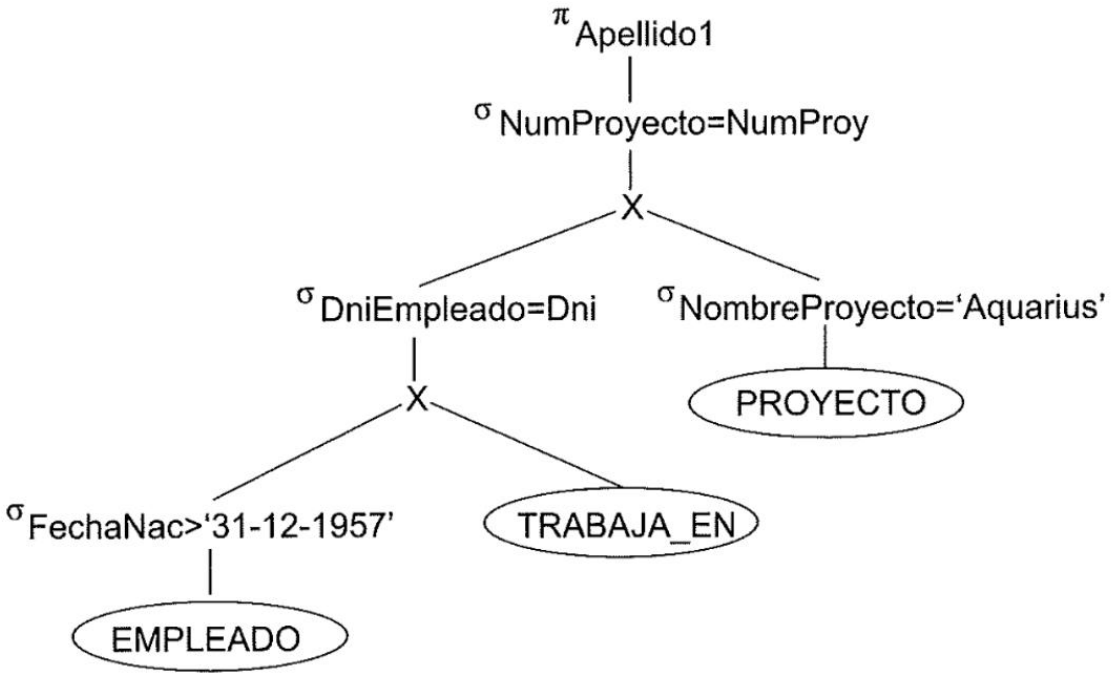
EMPLEADO(Dni,Nombre,Apellido1,Apellido2,FechaNac,Dirección,Sexo,Sueldo,…)
PROYECTO(NumProyecto,NombreProyecto,UbicacionProyecto,NumDeptoProyecto)
TRABAJA_EN(DniEmpleado,NumProy,Horas)

SELECT Apellido1
FROM EMPLEADO, TRABAJA_EN, PROYECTO
WHERE NombreProyecto="Aquarius" **AND** NumProyecto=NumProy **AND** DniEmpleado=Dni
AND FechaNac > "1957-12-31"

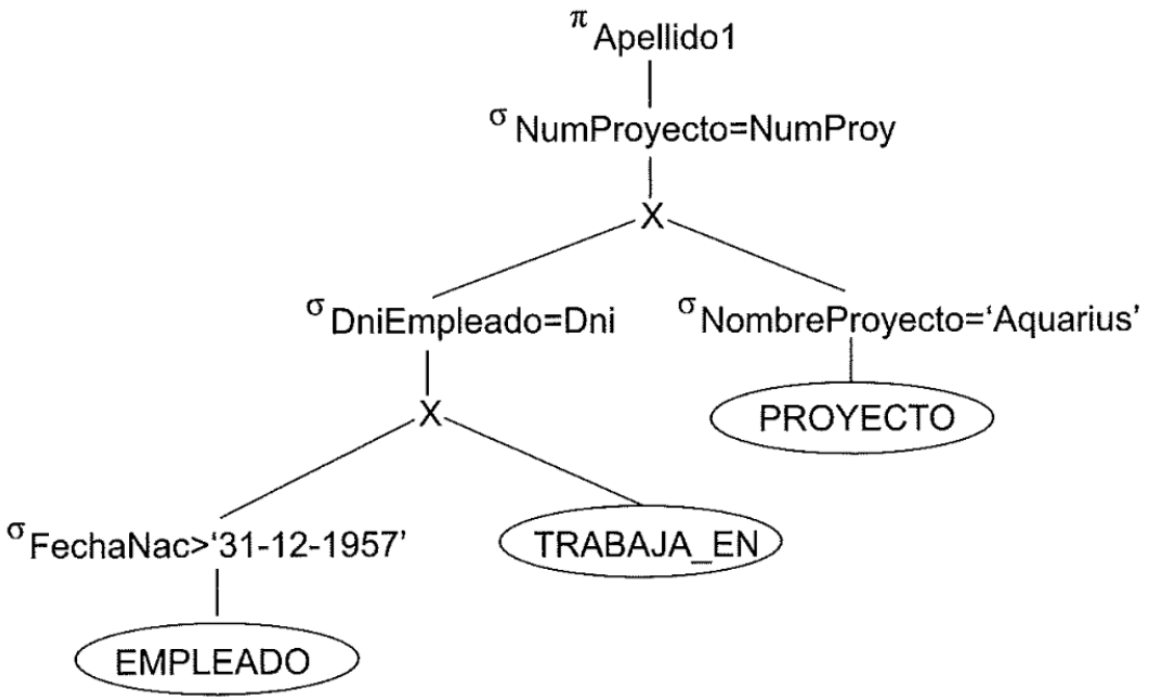


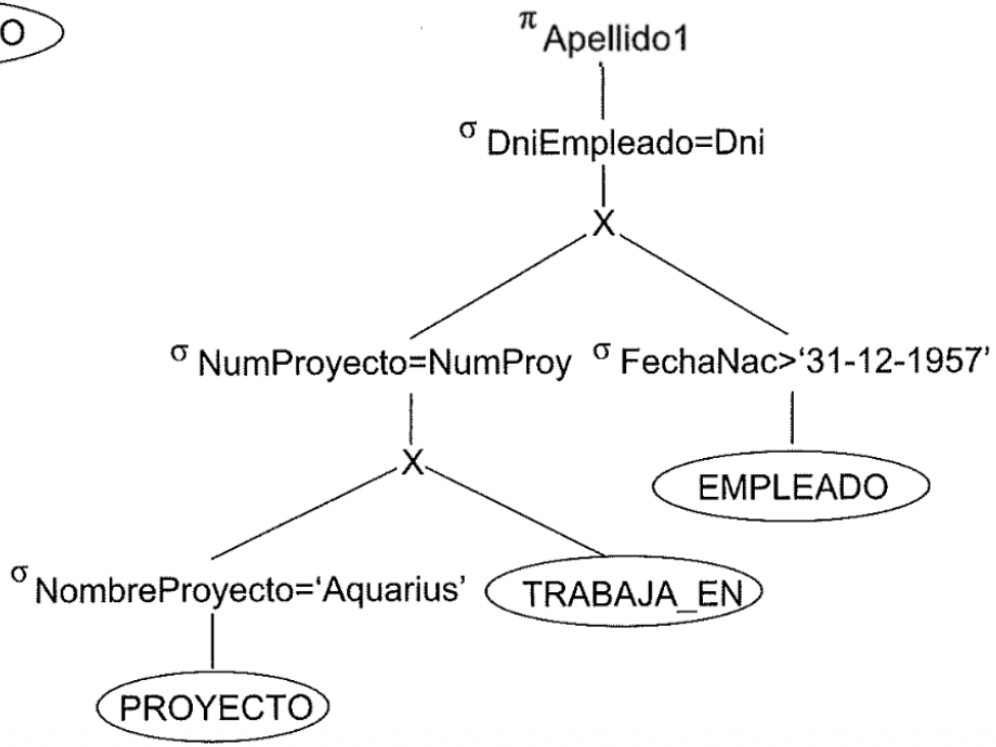
Ejemplo de transformación



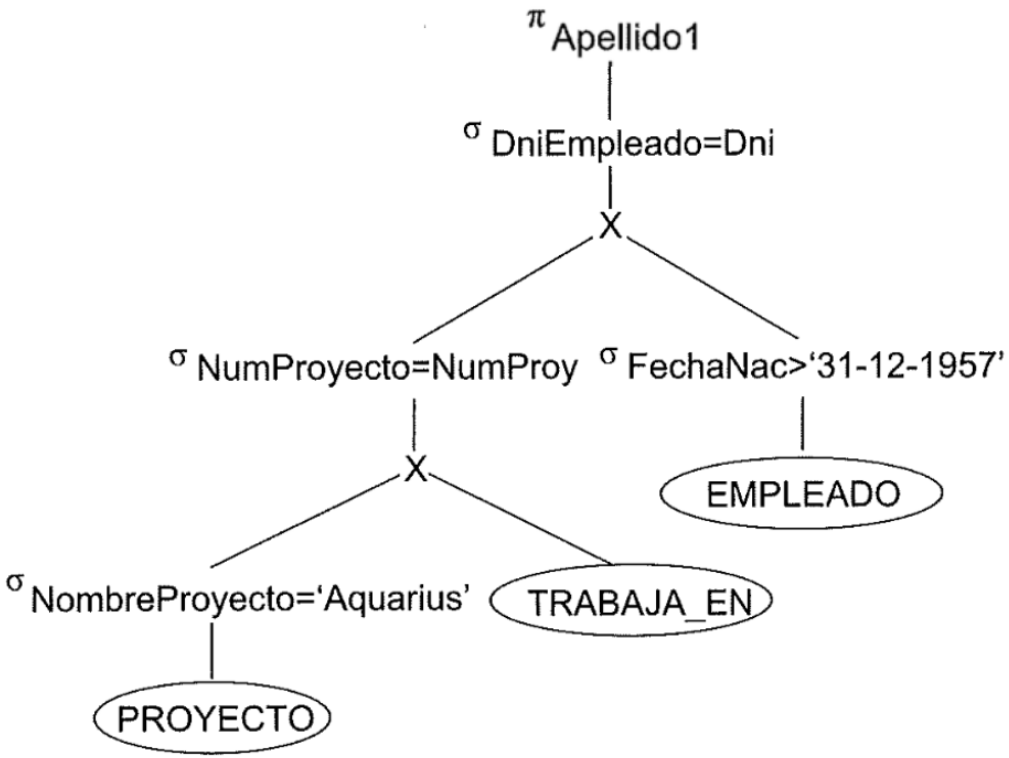


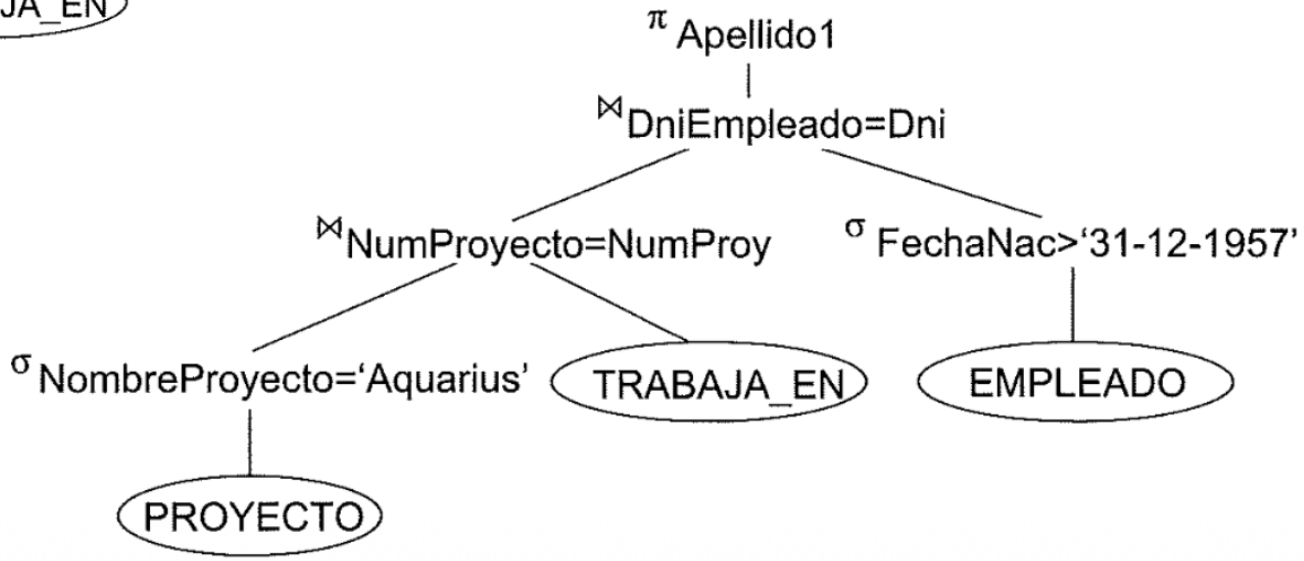
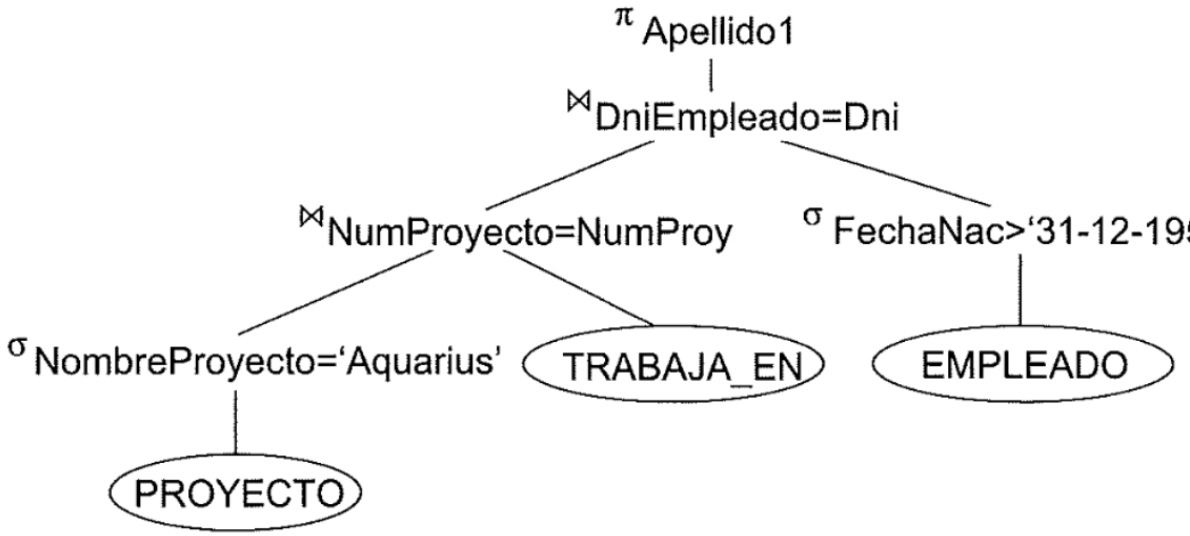
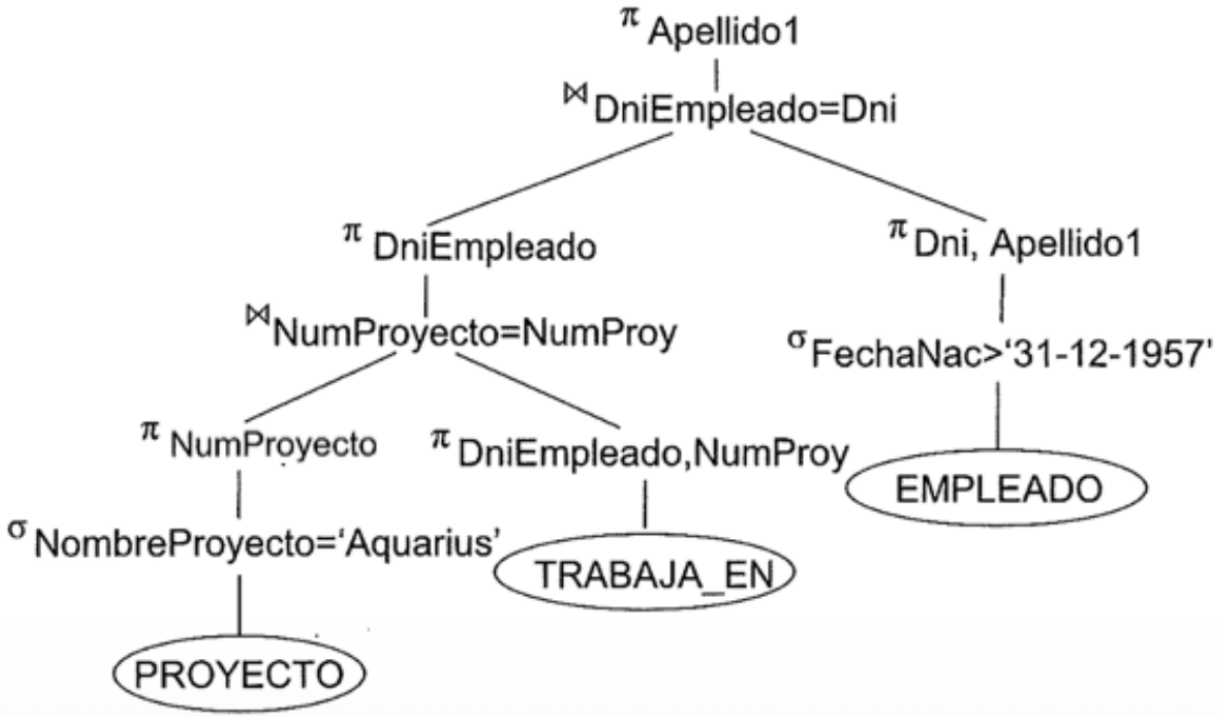
Ejemplo de transformación





Ejemplo de transformación





Ejemplo de transformación